

Deep Learning

Assignment 6

Build a Multiclass classifier using the CNN model. Use MNIST or any other suitable dataset.

- Perform Data Pre-processing
- Define Model and perform training
- Evaluate Results using confusion matrix

Name: Rohit Kalaskar

Roll No: 24

PRN: 12310106

Branch: CS AI - B

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

from sklearn.metrics import confusion_matrix, classification_report
```

```
(X_train, y_train), (X_test, y_test) = mnist.load_data()

print("Training shape:", X_train.shape)
print("Test shape:", X_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 — 0s 0us/step
Training shape: (60000, 28, 28)
Test shape: (10000, 28, 28)
```

```
# Normalize (0-255 → 0-1)
X_train = X_train / 255.0
X_test = X_test / 255.0

# Reshape to CNN format (28x28x1)
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)

# One-hot encoding
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)
```

```
model = Sequential()

# Convolution Layer
model.add(Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))

# Pooling Layer
model.add(MaxPooling2D(pool_size=(2,2)))

# Flatten
model.add(Flatten())

# Fully Connected Layers
model.add(Dense(128, activation='relu'))
model.add(Dense(10, activation='softmax')) # 10 classes

# Compile
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input`
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
flatten (Flatten)	(None, 5408)	0
dense (Dense)	(None, 128)	692,352
dense_1 (Dense)	(None, 10)	1,290

Total params: 693,962 (2.65 MB)

Trainable params: 693,962 (2.65 MB)

Non-trainable params: 0 (0.00 B)

```
history = model.fit(X_train, y_train_cat,
                    epochs=5,
                    validation_data=(X_test, y_test_cat))
```

```
Epoch 1/5
1875/1875 ————— 45s 23ms/step - accuracy: 0.9561 - loss: 0.1490 - val_accuracy: 0.9801 - val_loss: 0.0597
Epoch 2/5
1875/1875 ————— 94s 29ms/step - accuracy: 0.9847 - loss: 0.0512 - val_accuracy: 0.9823 - val_loss: 0.0490
Epoch 3/5
1875/1875 ————— 70s 23ms/step - accuracy: 0.9894 - loss: 0.0334 - val_accuracy: 0.9841 - val_loss: 0.0508
Epoch 4/5
1875/1875 ————— 79s 21ms/step - accuracy: 0.9928 - loss: 0.0222 - val_accuracy: 0.9852 - val_loss: 0.0434
Epoch 5/5
1875/1875 ————— 43s 23ms/step - accuracy: 0.9953 - loss: 0.0144 - val_accuracy: 0.9856 - val_loss: 0.0457
```

```
test_loss, test_acc = model.evaluate(X_test, y_test_cat)
print("Test Accuracy:", test_acc)
```

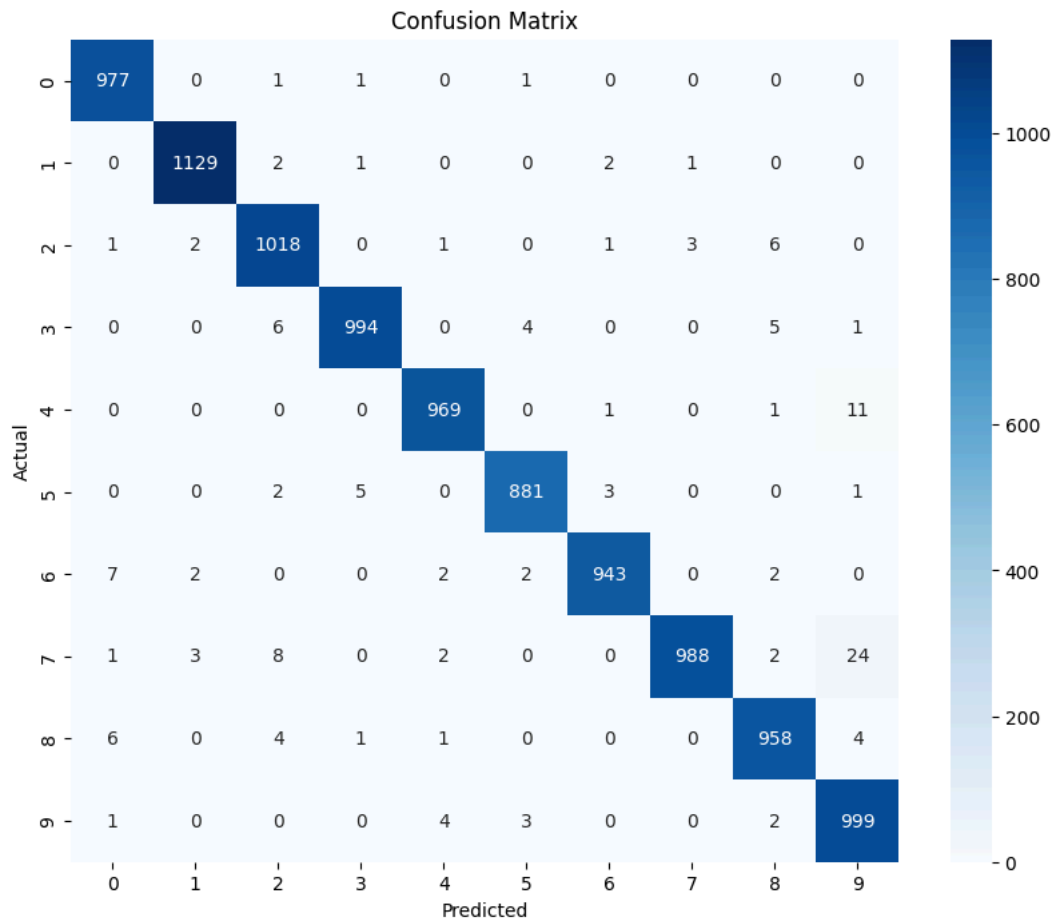
```
313/313 ————— 4s 12ms/step - accuracy: 0.9856 - loss: 0.0457
Test Accuracy: 0.9855999946594238
```

```
# Predictions
y_pred = model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred_classes)

# Plot
plt.figure(figsize=(10,8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

313/313 2s 6ms/step



```
print(classification_report(y_test, y_pred_classes))
```

	precision	recall	f1-score	support
0	0.98	1.00	0.99	980
1	0.99	0.99	0.99	1135
2	0.98	0.99	0.98	1032
3	0.99	0.98	0.99	1010
4	0.99	0.99	0.99	982
5	0.99	0.99	0.99	892
6	0.99	0.98	0.99	958
7	1.00	0.96	0.98	1028
8	0.98	0.98	0.98	974
9	0.96	0.99	0.98	1009